EDGE 1/29

CDN

Servers near each visitor that serve copies of your site's files

Local warehouses, so not every order ships from the factory

EDGE 2/29

Load balancer

Spreads requests across your server copies and skips broken ones

A traffic cop sending cars to open lanes

EDGE 3/29

Rate limits

Caps how often one person, bot or IP can hit you

A bouncer at the door

EDGE 4/29

Waiting room

Holds visitors on a "you're in line" page and lets them in at a set pace

A rope line outside a busy store

COMPUTE 5/29

Stateless API

Server copies remember nothing between requests, so any copy can serve anyone

Bank tellers: your account is in the system, not one teller's head

COMPUTE 6/29

Managed containers

The cloud runs copies of your packaged app, with no servers to manage

Renting a fitted kitchen instead of building one

COMPUTE 7/29

Autoscaling

Adds or removes copies as traffic, CPU or queue length changes

A thermostat: it reacts after the room gets hot

COMPUTE 8/29

Timeouts, retries, circuit breaker

Don't wait forever, retry politely, stop calling a dead service

Hang up on hold music, call back later, stop dialing if it's down

DATA 9/29

Postgres

The database that holds the truth and enforces its own rules

A shared spreadsheet that refuses bad edits

DATA 10/29

Object storage

Cheap storage for big files; the database keeps only the file name

Google Drive for your app

DATA 11/29

Cache

Keeps recent answers for a few seconds so the database can rest

A sticky note with today's specials, rewritten every 2 seconds

DATA 12/29

Replicas + failover

Database copies share the reads; a standby takes over if the main one dies

A co-pilot ready to take the controls

DATA 13/29

Connection pooler

Shares a few database connections among many server copies

A few phone lines shared by a whole office

ASYNC 14/29

Queue + workers

A to-do list for slow work; background workers do it later

Order tickets on a kitchen rail: the cashier doesn't cook

ASYNC 15/29

Dead-letter queue

Where a job goes after it fails too many times

The "problem orders" tray by the register

ASYNC 16/29

Hosted checkout + webhooks

Customers pay on the provider's page; the provider calls you when it worked

Letting the bank's teller handle the cash

CORRECTNESS 17/29

Conditional update

Check and change in one step: take one only if one is left

Only one hand can grab the last item on the shelf

CORRECTNESS 18/29

Holds + sweeper

Reserve first, pay second; a timed job frees holds nobody paid for

A store setting an item aside while you get your wallet

CORRECTNESS 19/29

Idempotency key

A unique id per action, so a repeat returns the first result

A numbered ticket: show it twice, get served once

CORRECTNESS 20/29

Webhook dedupe

Save each provider event id and ignore any you've already seen

Crossing off a delivery slip so it can't be paid twice

CORRECTNESS 21/29

State machine

A status that only moves forward: held, then paid or expired

A one-way street: no U-turns

CORRECTNESS 22/29

Reconciliation job

Compares your records with the outside truth and fixes the gaps

Counting the till against the receipts at closing

GO-LIVE 23/29

Load test (2-3x peak)

Fake a crowd 2 to 3 times your expected peak before the real one

A dress rehearsal before opening night

GO-LIVE 24/29

Pre-scale

Add capacity before a spike you know is coming

Calling in extra staff before the lunch rush

GO-LIVE 25/29

Kill switch / feature flag

Turn a feature on or off instantly, without a release

Light switches you install before the party

GO-LIVE 26/29

Canary + rollback

Release to a few users first, and undo fast if the numbers drop

Taste the soup before serving the whole room

GO-LIVE 27/29

Dashboard + alerts

Shows system health and wakes someone when a number looks wrong

A car dashboard with warning lights

GO-LIVE 28/29

Runbook + game day

Written "if this alarm, do this" steps, rehearsed before the event

A fire drill, with the exit map on the wall

GO-LIVE 29/29

Security basics

HTTPS, secrets in a vault, least access, and little personal data

Lock the doors, keep the keys in a safe, let the bank hold the cash

Framework: Custom Card Printing System

Flow
Understand the product or system

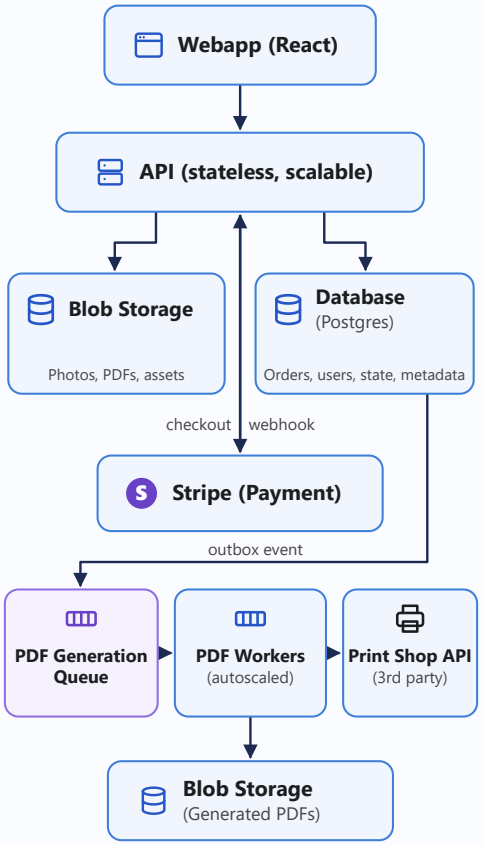
User flow
1. Upload photos
2. Choose template
3. Customize card (drag, text, etc.)
4. Preview card (≤ 30s)
5. Pay (Stripe)
6. Receive printed card (ship)

Requirements
- Normal traffic: 300 orders/day
- Peak: 15,000 orders/day (2,500/hr in the busiest hours)
- Preview latency: ≤ 30 seconds
- Correctness: no lost/duplicate paid orders

APIs (high level)
- `POST /uploads` (signed URL)
- `GET /templates`
- `POST /preview`
- `POST /orders` (pay)
- `GET /orders/{id}`

Data to store
- Photos (blobs)
- Templates
- Template drafts
- Orders (state, payment, print, shipping)

Core
Design the happy path



Bottleneck
What can break under peak load?

API & web tier
- Traffic spikes (2,500 orders/hr, several requests each)
- Keep stateless, autoscale, rate limit

Database (Postgres)
- High write volume (orders, state updates)
- Use proper indexing, connection pooling, read replicas (if needed)

Blob Storage
- Large file uploads and PDF writes
- Use retries, backoff, and concurrency limits

PDF generation
- Compute intensive, can backlog
- Use queue + autoscaling workers

Print shop API
- Third-party rate limits or slowness
- Add rate limiting, retries, and circuit breaker

Failures
How we handle failures for each component

Stripe succeeds, API crashes before saving
- Use Stripe webhooks (with retries)
- Sweeper + reconciliation: compare Stripe payments with our orders and create missing ones

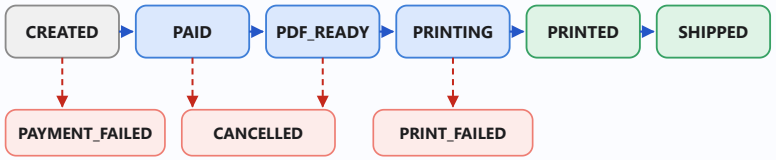
API saves payment, but doesn't enqueue PDF job
- Transactional outbox (write order + event in one txn)
- Sweeper to find PAID orders without a PDF job

Blob Storage outage (e.g., 20 minutes)
- Retries with exponential backoff
- Circuit breaker to pause retries
- Keep job in durable queue / DLQ for later retry

Large backlog after recovery (e.g., 20k jobs)
- Autoscale workers gradually
- Concurrency limits and rate limiting
- Process backlog without impacting new orders

Print shop prints but ack is lost
- Idempotency key per order (avoid duplicate prints)
- Reconciliation: periodically query the print shop and update order state

Order State Machine



Key Design Decisions
- Asynchronous PDF generation with a durable queue
- Idempotency keys (orders, Stripe, print shop)
- Transactional outbox + reconciliation sweeper
- Autoscaling and concurrency limits
- Circuit breaker & retry with exponential backoff
- State machine to track order lifecycle

Key Non-Functional Properties
- **Scalability:** handle up to 15k orders/day
- **Reliability:** no lost or duplicate orders
- **Performance:** preview within 30s
- **Consistency:** reconcile with Stripe and print shop
- **Availability:** continue taking orders during downstream outages
- **Observability:** logs, metrics, alerts, and job dashboards

End-to-End Happy Path (Completed Design)

User customizes & pays (≤ 30s preview) → Order recorded (+ outbox event) → PDF generated (async worker) → Sent to print shop (idempotent) → Printed & shipped to customer